

LAPORAN POSTTEST 4
PRAKTIKUM BASIS DATA



Disusun oleh:
Anugerah Fakhriza Reswara (2509106025)
Kelas (A2'25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA

2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB 1 PENJELASAN POSTTEST 4.....	1
1.1 Penjelasan.....	1
1.2 Tujuan.....	1
BAB 2 PENJELASAN ENTITAS DAN ATRIBUT.....	3
2.1 Entitas dan Atribut.....	3
2.1.1 Entitas Pelanggan (Tabel: pelanggan).....	3
2.1.2 Entitas Transaksi (Tabel: transaksi).....	3
BAB 3 IMPLEMENTASI DAN ANALISIS HASIL.....	5
3.1 Inisialisasi dan persiapan data.....	5
3.1.1 Pembuatan Skema Database.....	5
3.1.2 Relasi Tamu - Pemesanan (One-to-Many / 1:N).....	5
3.1.3 Pengisian Data Sampel (Seeding).....	6
3.1.4 Output setelah pengisian data sampel.....	7
3.2 Analisis NOT IN dengan Subquery.....	8
3.2.1 Filter Berdasarkan Saldo Pelanggan.....	8
3.2.1 Filter Berdasarkan Nominal Transaksi.....	8
3.3 Implementasi ORDER BY Descending dan Filter Logika.....	9
3.3.1 Kondisi Umur < 30 Tahun.....	9
3.3.2 Filter Tanggal Bergabung (Historical Data).....	10
3.4 Implementasi ALL.....	10
3.5 Penggabungan Data dengan UNION.....	11
3.6 Manipulasi String Lanjutan dengan NOT LIKE.....	12
3.6.1 Validasi Struktur Data (Panjang Karakter).....	12
3.6.2 Penyaringan Berdasarkan Inisial (Huruf Awal).....	12
3.6.3 Pencarian Konten Spesifik (Huruf Tengah/Bagian Kata).....	13

3.6.4 Analisis Akhiran Kata (Huruf Akhir).....	13
BAB 4 KESIMPULAN.....	15
4.1 Ringkasan Teknis.....	15
4.2. Nilai Strategis Penguasaan Query.....	15
BAB 5 PENUTUP.....	16
5.1 Penutup.....	16
5.2 Saran Pengembangan.....	16

BAB 1

PENJELASAN POSTTEST 4

1.1 Penjelasan

Dalam era transformasi digital, data telah menjadi aset yang sangat berharga bagi setiap organisasi. Namun, data mentah yang tersimpan dalam jumlah besar tidak akan memberikan manfaat maksimal tanpa adanya kemampuan untuk memproses dan menganalisisnya secara efisien. Basis data relasional muncul sebagai solusi untuk mengorganisir informasi secara sistematis, memungkinkan akses cepat, pengelolaan yang terstruktur, dan pembaruan data yang konsisten.

SQL (Structured Query Language) adalah bahasa standar yang digunakan untuk berinteraksi dengan Sistem Manajemen Basis Data Relasional (RDBMS). Penguasaan SQL tidak hanya terbatas pada perintah dasar seperti Create, Read, Update, dan Delete (CRUD), tetapi juga mencakup teknik manipulasi data tingkat lanjut. Penggunaan subquery, operator set, dan filter string yang kompleks sangat krusial dalam dunia kerja nyata, misalnya untuk menghasilkan laporan keuangan, analisis perilaku pelanggan, hingga optimasi stok barang yang memerlukan penggabungan informasi dari berbagai sumber tabel secara logis dan efisien.

1.2 Tujuan

Tujuan utama dari posttest 4 ini adalah untuk memberikan pemahaman praktis serta pengalaman langsung dalam:

- Mengimplementasikan perintah NOT IN dikombinasikan dengan teknik subquery untuk menyaring data berdasarkan kriteria yang dinamis dari tabel lain.
- Menerapkan mekanisme pengurutan data (Ordering) secara descending dengan filter logika tingkat lanjut guna menyajikan data yang informatif.
- Memahami dan mempraktikkan penggunaan operator ALL untuk perbandingan data skalar terhadap kumpulan nilai (result set).

- Menganalisis penggunaan operator UNION untuk menggabungkan dua atau lebih hasil query menjadi satu kesatuan informasi yang utuh.
- Melakukan eksplorasi filter teks menggunakan operator NOT LIKE dengan berbagai pola wildcard guna melakukan pencarian negatif atau pembersihan data dari pola yang tidak diinginkan.

BAB 2

PENJELASAN ENTITAS DAN ATRIBUT

2.1 Entitas dan Atribut

Entitas adalah komponen utama pada level Logical Model yang merepresentasikan objek atau konsep di dunia nyata yang informasinya ingin disimpan sedangkan Atribut adalah elemen data yang mendefinisikan karakteristik atau properti spesifik dari sebuah entitas.

2.1.1 Entitas Pelanggan (Tabel: pelanggan)

Tabel ini berfungsi sebagai master data yang menyimpan informasi fundamental mengenai individu yang terdaftar dalam sistem. Berikut adalah atribut-atribut yang ada di tabel pelanggan:

- **id (VARCHAR 5):** Bertindak sebagai Primary Key. Penggunaan tipe data VARCHAR memberikan fleksibilitas jika identitas menggunakan kombinasi huruf dan angka.
- **nama (VARCHAR 50):** Menyimpan nama lengkap. Atribut NOT NULL memastikan setiap entitas wajib memiliki identitas nama.
- **alamat (VARCHAR 100):** Informasi domisili untuk keperluan logistik atau korespondensi.
- **umur (INT):** Data numerik usia yang digunakan untuk segmentasi pasar atau analisis demografi.
- **saldo (DECIMAL 10,2):** Menyimpan nilai finansial. Tipe Decimal digunakan untuk menghindari isu presisi pada angka pecahan yang sering terjadi pada tipe Float.
- **tanggal_bergabung (DATE):** Rekam jejak loyalitas pelanggan sejak pertama kali terdaftar.

2.1.2 Entitas Transaksi (Tabel: transaksi)

Tabel ini mencatat setiap histori pembelian yang terjadi, mencerminkan aktivitas ekonomi dalam sistem. Berikut adalah atribut-atribut yang ada di tabel transaksi:

- **id (VARCHAR 10):** Identitas unik setiap transaksi (misal: kode struk).
- **total_pembelian (DECIMAL 12,2):** Nilai belanja bruto dalam satu sesi transaksi.
- **waktu_transaksi (TIMESTAMP):** Mencatat waktu secara presisi hingga satuan detik, memudahkan

analisis waktu puncak (peak hour) transaksi.

BAB 3

IMPLEMENTASI DAN ANALISIS HASIL

3.1 Inisialisasi dan persiapan data

Proses implementasi dimulai dengan tahap krusial yaitu membangun lingkungan fisik database melalui perintah Data Definition Language (DDL). Langkah ini tidak hanya sekedar membuat wadah penyimpanan, tetapi juga menetapkan aturan main atau business rules yang harus dipatuhi oleh setiap data yang akan masuk ke dalam sistem.

3.1.1 Pembuatan Skema Database

Perintah CREATE DATABASE digunakan untuk mengalokasikan ruang nama (namespace) terisolasi di dalam RDBMS. Hal ini memastikan bahwa tabel-tabel yang dibuat tidak akan berbenturan dengan proyek lain dalam satu server yang sama. Penggunaan perintah USE setelahnya sangat penting untuk menetapkan konteks kerja aktif, sehingga instruksi selanjutnya dieksekusi pada basis data yang tepat.

3.1.3 Pengisian Data Sampel (Seeding)

Tahap terakhir dalam persiapan adalah pengisian data awal menggunakan perintah Data Manipulation Language (DML) yaitu INSERT INTO. Data yang dimasukkan dirancang sedemikian rupa untuk mencakup berbagai skenario pengujian, seperti variasi umur, rentang saldo yang ekstrem (dari ribuan hingga puluhan juta), serta tanggal bergabung yang tersebar untuk menguji validitas query pencarian berdasarkan waktu.

```
-- Membuat database baru dan menggunakannya
CREATE DATABASE POSTTEST_4;
USE POSTTEST_4;

-- Mendefinisikan struktur tabel pelanggan
CREATE TABLE pelanggan (
    id VARCHAR (5) PRIMARY KEY,
    nama VARCHAR (50) NOT NULL,
    alamat VARCHAR (100),
    umur INT NOT NULL,
    saldo DECIMAL(10,2) NOT NULL,
```



```

        tanggal_bergabung DATE NOT NULL
    );

-- Mendefinisikan struktur tabel transaksi
CREATE TABLE transaksi (
    id VARCHAR(10) PRIMARY KEY,
    total_pembelian DECIMAL(12,2) NOT NULL,
    waktu_transaksi TIMESTAMP NOT NULL
);

-- Memasukkan data sampel ke tabel pelanggan
INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES
    ('1', 'Anugerah Fakhri Reswara', 'Jalan In Aja', 18, 10000.00,
    '2025-08-01');

INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES
    ('2', 'Muhammad Alfauzi Syahputra', 'Jalan Harapan Baru', 10,
    50000000.00, '2023-02-11');

INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES
    ('3', 'Pirlo Syabila Hafuza', 'Jalan Praktek', 24, 110000.00,
    '2024-04-02');

INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES
    ('4', 'Yoga Ananda Prasetya', 'Jalan Teori', 11, 220000.00,
    '2025-05-20');

INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES
    ('5', 'Akbar Rachim', 'Jalan Yang Diatas', 19, 500000.00,
    '2026-01-31');

```

3.1.4 Output setelah pengisian data sampel

Setelah dilakukannya pengisian sampel pada 2 tabel tersebut, untuk melihat data yang sudah dimasukkan, saya akan menggunakan query `SELECT * FROM namatabel;`

```
SELECT * FROM pelanggan;
```

	id	nama	alamat	umur	saldo	tanggal_bergabung
1	1	Anugerah Fakhriza Reswara	Jalan In Aja	18	10000.00	2025-08-01
2	2	Muhammad Alfauzi Syahputra	Jalan Harapan Baru	10	50000000.00	2023-02-11
3	3	Pirlo Syabila Hafuza	Jalan Praktek	24	110000.00	2024-04-02
4	4	Yoga Ananda Prasetya	Jalan Teori	11	220000.00	2025-05-20
5	5	Akbar Rachim	Jalan Yang Diatas	19	500000.00	2026-01-31
6	6	Denny Mulia	Jalan Sepuh	45	71500.00	2022-12-15
7	7	Muhammad Zidane Abdul Kadir	Jalan Luas	50	923010.00	2021-05-20

Gambar 3.1.4.1 Isi Data Dari Tabel Pelanggan

```
SELECT * FROM transaksi;
```

	id	total_pembelian	waktu_transaksi
1	ORDER001	200000.00	2025-11-22 08:33:44
2	ORDER002	150241.00	2025-11-04 21:22:33
3	ORDER003	56040.00	2026-02-02 15:30:23
4	ORDER004	44420.00	2026-03-04 14:11:00
5	ORDER005	190931.00	2026-03-15 22:55:21

Gambar 3.1.4.2 Isi Data Dari Tabel Transaksi

3.2 Analisis NOT IN dengan Subquery

Teknik Subquery memungkinkan kita untuk menjalankan query di dalam query lainnya. Perintah NOT IN di sini digunakan untuk mengecualikan baris data yang memenuhi kondisi tertentu yang dihasilkan oleh subquery tersebut.

3.2.1 Filter Berdasarkan Saldo Pelanggan

Dalam kasus ini, subquery mencari semua nilai saldo di atas 150.000 (yaitu saldo milik pelanggan '2', '4', dan '5'). Query utama kemudian mengambil semua data pelanggan yang saldonya tidak ada dalam daftar tersebut. Hasilnya adalah daftar pelanggan menengah ke bawah yaitu pelanggan dengan id '1' dan '3'.

```
SELECT * FROM pelanggan WHERE saldo NOT IN (SELECT saldo FROM pelanggan
WHERE saldo > 150000.00);
```

	id	nama	alamat	umur	saldo	tanggal_bergabung
1	1	Anugerah Fakhriza Reswara	Jalan In Aja	18	10000.00	2025-08-01
2	3	Pirlo Syabila Hafuza	Jalan Praktek	24	110000.00	2024-04-02

Gambar 3.2.1 Output Query Filter Saldo

3.2.1 Filter Berdasarkan Nominal Transaksi

Selain pada data master pelanggan, perintah NOT IN juga diterapkan pada data transaksi untuk mengklasifikasikan nilai pembelian. Query ini bekerja dengan cara:

1. **Eksekusi Subquery:** Sistem pertama-tama mencari semua baris dalam tabel transaksi yang memiliki

total_pembelian kurang dari 100.000,00 (dalam data sampel kita, ini mencakup ORDER003 dan ORDER004).

2. **Pembentukan List:** Hasil dari subquery tersebut berupa sekumpulan nilai desimal (\$56.040,00\$, \$44.420,00\$).
3. **Filtrasi Utama:** Query luar akan mengevaluasi setiap baris di tabel transaksi. Jika total_pembelian suatu baris tidak ada dalam daftar (list) yang dihasilkan subquery tadi, maka baris tersebut akan ditampilkan.
4. **Implikasi:** Hasil akhir dari query ini secara efektif menampilkan "Transaksi Bernilai Tinggi" (High-Value Transactions), yaitu transaksi dengan nominal 100.000,00 ke atas. Pendekatan ini sangat berguna dalam audit finansial untuk memisahkan transaksi mikro dari transaksi utama tanpa perlu mengubah struktur tabel.

```
SELECT * FROM transaksi WHERE total_pembelian NOT IN (SELECT  
total_pembelian FROM transaksi WHERE total_pembelian < 100000.00);
```

	id	total_pembelian	waktu_transaksi
1	ORDER001	200000.00	2025-11-22 08:33:44
2	ORDER002	150241.00	2025-11-04 21:22:33
3	ORDER005	190931.00	2026-03-15 22:55:21

Gambar 3.2.2 Output Query Filter Berdasarkan Nominal Transaksi

3.3 Implementasi ORDER BY Descending dan Filter Logika

Pengurutan data sangat penting dalam presentasi laporan agar informasi yang paling relevan muncul di posisi paling atas.

3.3.1 Kondisi Umur < 30 Tahun

Perintah ini sangat efektif untuk segmentasi audiens muda. Pengurutan DESC memastikan kita melihat pelanggan tertua di dalam rentang umur tersebut terlebih dahulu.

```
SELECT * FROM pelanggan WHERE umur < 30 ORDER BY umur DESC;
```

	id	nama	alamat	umur	saldo	tanggal_bergabung
1	3	Pirlo Syabila Hafuza	Jalan Praktek	24	110000.00	2024-04-02
2	5	Akbar Rachim	Jalan Yang Diatas	19	500000.00	2026-01-31
3	1	Anugerah Fakhriza Reswara	Jalan In Aja	18	10000.00	2025-08-01
4	4	Yoga Ananda Prasetya	Jalan Teori	11	220000.00	2025-05-20
5	2	Muhammad Alfauzi Syahputra	Jalan Harapan Baru	10	50000000.00	2023-02-11

Gambar 3.3.1.1 Output Query Filter Pengurutan Descending Berdasarkan Umur

3.3.2 Filter Tanggal Bergabung (Historical Data)

Karena data awal tidak ada yang sebelum Januari 2023, dilakukan penambahan record baru untuk memvalidasi query. Hasil ini membantu perusahaan mengidentifikasi "pelanggan setia" yang bergabung di masa lampau.

```
INSERT INTO pelanggan (id, nama, alamat, umur, saldo, tanggal_bergabung)
VALUES
('6', 'Denny Mulia', 'Jalan Sepuh', 45, 71500.00, '2022-12-15'),
('7', 'Muhammad Zidane Abdul Kadir', 'Jalan Luas', 50, 923010.00,
'2021-05-20');

-- Query pencarian pelanggan lama
SELECT * FROM pelanggan
WHERE tanggal_bergabung < '2023-01-01'
ORDER BY tanggal_bergabung DESC;
```

	id	nama	alamat	umur	saldo	tanggal_bergabung
1	6	Denny Mulia	Jalan Sepuh	45	71500.00	2022-12-15
2	7	Muhammad Zidane Abdul Kadir	Jalan Luas	50	923010.00	2021-05-20

Gambar 3.3.2.1 Output Query Filter Pengurutan Descending Berdasarkan Pelanggan Setia

3.4 Implementasi ALL

Operator ALL membandingkan nilai kolom dengan setiap nilai tunggal yang dihasilkan oleh subquery. Jika subquery transaksi menghasilkan nilai

44k, 56k, 150k, 190k, 200k

, maka query utama hanya akan menampilkan pelanggan yang saldonya lebih besar dari nilai maksimum di daftar tersebut (yaitu > 200000.00).

```
SELECT * FROM pelanggan WHERE saldo > ALL (SELECT total_pembelian
FROM transaksi);
```

	id	nama	alamat	umur	saldo	tanggal_bergabung
1	2	Muhammad Alfauzi Syahputra	Jalan Harapan Baru	10	50000000.00	2023-02-11
2	4	Yoga Ananda Prasetya	Jalan Teori	11	220000.00	2025-05-20
3	5	Akbar Rachim	Jalan Yang Diatas	19	500000.00	2026-01-31
4	7	Muhammad Zidane Abdul Kadir	Jalan Luas	50	923010.00	2021-05-20

Gambar 3.4.1 Output Query ALL

3.5 Penggabungan Data dengan UNION

Operator UNION digunakan untuk menggabungkan dua set hasil query SELECT yang memiliki jumlah dan tipe kolom yang kompatibel. Operasi ini menghasilkan satu daftar tunggal berisi semua ID yang ada di sistem, baik itu ID manusia (pelanggan) maupun ID dokumen (transaksi). Secara otomatis, UNION akan menghapus duplikasi jika ditemukan ID yang sama di kedua tabel tersebut.

```
SELECT id AS 'ID_Gabungan' FROM pelanggan UNION SELECT id FROM transaksi;
```

	ID_Gabungan
1	1
2	2
3	3
4	4
5	5
6	6
7	7

Gambar 3.5.1 Output Query UNION

3.6 Manipulasi String Lanjutan dengan NOT LIKE

Operator NOT LIKE adalah instrumen yang sangat berdaya dalam SQL untuk melakukan penyaringan data berbasis teks secara negatif. Berbeda dengan LIKE yang mencari kecocokan pola, NOT LIKE mengecualikan semua data yang memenuhi pola tertentu. Fleksibilitas ini didukung oleh dua wildcard utama: persen (%) yang mewakili urutan karakter apa pun (termasuk nol karakter) dan garis bawah (_) yang mewakili tepat satu karakter tunggal.

3.6.1 Validasi Struktur Data (Panjang Karakter)

Penggunaan karakter garis bawah (_) secara berulang memungkinkan kita untuk memvalidasi panjang string secara spesifik. Pola _____ akan cocok dengan string seperti 'Akbar'. Dengan NOT LIKE, kita mengecualikan nama-nama pendek tersebut. Teknik ini sangat krusial dalam proses Data Cleansing (Pembersihan Data) untuk mendeteksi input nama yang mungkin tidak valid atau terlalu singkat (seperti inisial saja).

```
SELECT * FROM pelanggan WHERE nama NOT LIKE '_____';
```

	id	nama	alamat	umur	saldo	tanggal_bergabung
1	1	Anugerah Fakhriza Reswara	Jalan In Aja	18	10000.00	2025-08-01
2	2	Muhammad Alfauzi Syahputra	Jalan Harapan Baru	10	50000000.00	2023-02-11
3	3	Pirlo Syabila Hafuza	Jalan Praktek	24	110000.00	2024-04-02
4	4	Yoga Ananda Prasetya	Jalan Teori	11	220000.00	2025-05-20
5	5	Akbar Rachim	Jalan Yang Diatas	19	500000.00	2026-01-31
6	6	Denny Mulia	Jalan Sepuh	45	71500.00	2022-12-15
7	7	Muhammad Zidane Abdul Kadir	Jalan Luas	50	923010.00	2021-05-20

Gambar 3.6.1.1 Output Query Panjang Karakter

3.6.2 Penyaringan Berdasarkan Inisial (Huruf Awal)

Wildcard % yang diletakkan setelah karakter tertentu akan mencari semua kata yang dimulai dengan karakter tersebut. Dalam skenario dunia nyata, query ini berguna untuk melakukan segmentasi beban kerja. Misalnya, jika sebuah departemen hanya menangani pelanggan berinisial 'A', departemen lain dapat menggunakan query ini untuk melihat sisa pelanggan lainnya tanpa terganggu oleh data yang sudah ditangani.

```
SELECT * FROM pelanggan WHERE nama NOT LIKE 'A%';
```

	id	nama	alamat	umur	saldo	tanggal_bergabung
1	2	Muhammad Alfauzi Syahputra	Jalan Harapan Baru	10	50000000.00	2023-02-11
2	3	Pirlo Syabila Hafuza	Jalan Praktek	24	110000.00	2024-04-02
3	4	Yoga Ananda Prasetya	Jalan Teori	11	220000.00	2025-05-20
4	6	Denny Mulia	Jalan Sepuh	45	71500.00	2022-12-15
5	7	Muhammad Zidane Abdul Kadir	Jalan Luas	50	923010.00	2021-05-20

Gambar 3.6.2.1 Output Query Bagian Awal

3.6.3 Pencarian Konten Spesifik (Huruf Tengah/Bagian Kata)

Meletakkan wildcard % di kedua sisi kata akan mencari keberadaan kata tersebut di posisi mana pun dalam string. Pola ini sering digunakan dalam sistem keamanan atau penyaringan konten (content filtering). Misalnya, untuk menyaring nama-nama yang mengandung kata-kata terlarang atau untuk mengecualikan sub-keluarga tertentu dalam database kependudukan. Ini menjamin akurasi tinggi karena posisi kata 'Ananda' di awal, tengah, maupun akhir nama akan otomatis terdeteksi untuk dikecualikan.

```
SELECT * FROM pelanggan WHERE nama NOT LIKE '%Ananda%';
```

	id	nama	alamat	umur	saldo	tanggal_bergabung
1	1	Anugerah Fakhriza Reswara	Jalan In Aja	18	10000.00	2025-08-01
2	2	Muhammad Alfauzi Syahputra	Jalan Harapan Baru	10	50000000.00	2023-02-11
3	3	Pirlo Syabila Hafuza	Jalan Praktek	24	110000.00	2024-04-02
4	5	Akbar Rachim	Jalan Yang Diatas	19	500000.00	2026-01-31
5	6	Denny Mulia	Jalan Sepuh	45	71500.00	2022-12-15
6	7	Muhammad Zidane Abdul Kadir	Jalan Luas	50	923010.00	2021-05-20

Gambar 3.6.3.1 Output Query Bagian Tengah

3.6.4 Analisis Akhiran Kata (Huruf Akhir)

Wildcard % yang diletakkan di depan karakter target akan memfokuskan pencarian pada akhir dari sebuah string. Meskipun terlihat sederhana, filter ini memiliki kegunaan linguistik atau kategorisasi otomatis. Dalam beberapa budaya, akhiran nama dapat memberikan petunjuk kasar mengenai gender atau asal daerah. Penggunaan NOT LIKE '%a' memungkinkan analisis untuk dengan cepat memisahkan kelompok data berdasarkan karakteristik fonetik akhiran nama untuk keperluan statistik awal.

```
SELECT * FROM pelanggan WHERE nama NOT LIKE '%a';
```


	id	nama	alamat	umur	saldo	tanggal_bergabung
1	5	Akbar Rachim	Jalan Yang Diatas	19	500000.00	2026-01-31
2	7	Muhammad Zidane Abdul Kadir	Jalan Luas	50	923010.00	2021-05-20

Gambar 3.6.4.1 Output Query Bagian Akhir

BAB 4

KESIMPULAN

4.1 Ringkasan Teknis

Berdasarkan hasil praktikum yang telah dilaksanakan, dapat disimpulkan bahwa SQL merupakan instrumen yang jauh lebih kompleks daripada sekadar media penyimpanan data statis. Implementasi query lanjutan memberikan dimensi baru dalam analisis data relasional:

Analisis Relasional: Penggunaan teknik subquery dengan operator NOT IN dan ALL terbukti sangat efektif untuk melakukan perbandingan dinamis antar entitas. Hal ini memungkinkan pengambilan keputusan berbasis data yang lebih presisi, seperti mengidentifikasi pelanggan bernilai tinggi atau memantau anomali transaksi.

Efisiensi Penyajian: Mekanisme pengurutan data (Ordering) dan penggabungan set melalui UNION memfasilitasi pembuatan laporan yang komprehensif tanpa perlu melakukan pengolahan data manual di aplikasi luar basis data.

Integritas dan Validasi: Penggunaan operator NOT LIKE menunjukkan fleksibilitas luar biasa dalam menjaga kebersihan data (data cleansing) serta melakukan segmentasi demografis berbasis pola teks yang sulit dilakukan dengan filter numerik biasa.

4.2. Nilai Strategis Penguasaan Query

Secara keseluruhan, penguasaan teknik-teknik ini merupakan fondasi utama bagi seorang pengembang aplikasi maupun analis data. Kemampuan untuk merumuskan query kompleks secara efisien berbanding lurus dengan kemampuan sistem dalam menyajikan informasi yang akurat, cepat, dan relevan dalam lingkungan sistem informasi yang dinamis.

BAB 5

PENUTUP

5.1 Penutup

Laporan praktikum ini telah mendokumentasikan setiap tahapan pengerjaan tugas basis data, mulai dari perancangan skema tabel hingga eksekusi query manipulasi tingkat lanjut. Melalui pengerjaan ini, praktikan berhasil mencapai seluruh tujuan pembelajaran yang ditetapkan, terutama dalam memahami logika di balik operator-operator SQL yang lebih lanjut. Keberhasilan implementasi query ini memberikan gambaran nyata tentang bagaimana basis data dapat dioptimalkan untuk mendukung kebutuhan operasional maupun manajerial dalam suatu organisasi.

5.2 Saran Pengembangan

Untuk pengembangan lebih lanjut, disarankan agar:

- **Optimasi Indexing:** Dilakukan penerapan indexing pada kolom-kolom yang sering dijadikan filter dalam query NOT LIKE atau ALL untuk menjaga performa sistem saat volume data meningkat.
- **Eksplorasi Join:** Menggabungkan teknik subquery yang telah dipelajari dengan perintah JOIN yang lebih kompleks untuk mendapatkan wawasan data yang lebih integratif antar tabel.
- **Keamanan Data:** Mulai mempertimbangkan implementasi Stored Procedures atau Views untuk mengemas query-query kompleks ini agar lebih aman dan mudah dipanggil kembali oleh aplikasi.